

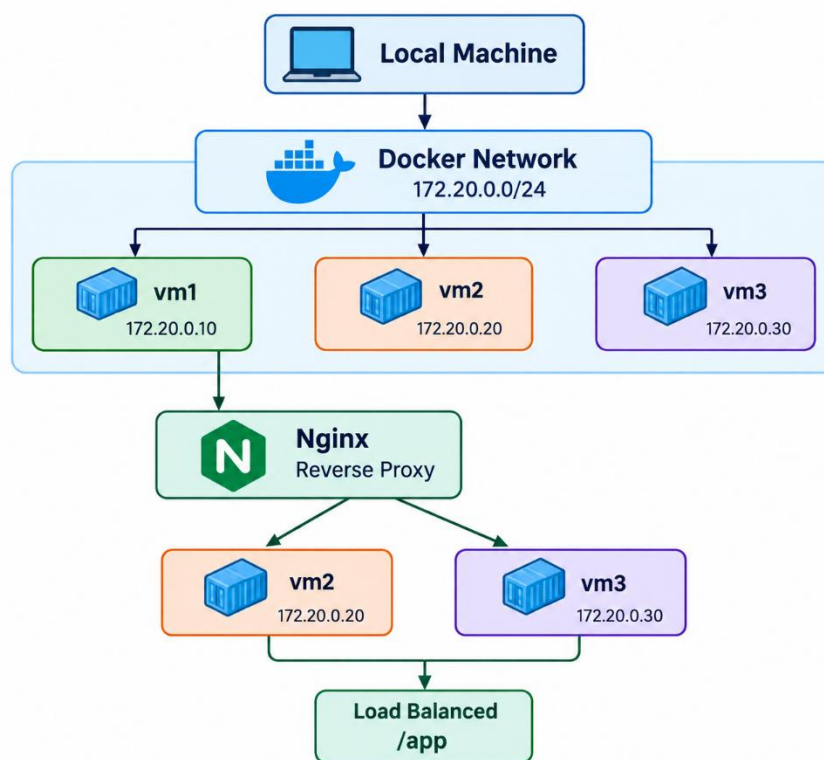
# Docker-Based Virtual Infrastructure with Ansible Automation and Hardened Nginx Reverse Proxy

## 1. Project Overview

### Objective

The objective of this project is to create a local virtual infrastructure using Docker containers as VM stand-ins. Three Ubuntu containers — vm1, vm2, and vm3 — are configured with SSH, UFW firewall, Ansible automation, Nginx, HTTPS, reverse proxying, load balancing, and failover.

### Architecture:



## 2. Docker Setup

### 2.1 Docker Installation

Docker Desktop was installed on the Windows host machine. WSL 2 integration was used to provide a Linux environment for Ansible and project management.

### Verify Docker:

```
docker --version
```

### Verify containers:

## docker ps

```
adhit@LAPTOP-N3NM26QJ MINGW64 ~/docker-ansible-nginx-assessment
$ docker ps
```

| CONTAINER ID | IMAGE             | COMMAND             | CREATED       | STATUS       | PORTS | NAMES |
|--------------|-------------------|---------------------|---------------|--------------|-------|-------|
| 638908e022ef | devops-ubuntu:1.0 | "/usr/sbin/sshd -D" | 2 minutes ago | Up 2 minutes |       | vm3   |
| 0dec60689f2c | devops-ubuntu:1.0 | "/usr/sbin/sshd -D" | 2 minutes ago | Up 2 minutes |       | vm2   |
| 5f5315d8ea6  | devops-ubuntu:1.0 | "/usr/sbin/sshd -D" | 3 minutes ago | Up 3 minutes |       | vm1   |

running containers

## 2.2 Create Custom Docker Network

A custom Docker network was created instead of using Docker's default network.

The network uses:

Network: devops\_net

Subnet: 172.20.0.0/24

Verify:

## docker network inspect devops\_net

The three containers are connected to this network.

```
MINGW64/c/Users/adhit/docker-ansible-nginx-assessment
adhit@LAPTOP-N3NM26QJ MINGW64 ~/docker-ansible-nginx-assessment
$ docker network ls
NETWORK ID          NAME                DRIVER              SCOPE
7ae79d2f411b        bridge              bridge              local
63aea7625dca        devops_net          bridge              local
e1922caddab2        host                host                local
485b846b8ac2        kind                bridge              local
55bb22a67561        none                null                local

adhit@LAPTOP-N3NM26QJ MINGW64 ~/docker-ansible-nginx-assessment
$ docker network inspect devops_net
[
  {
    "Name": "devops_net",
    "Id": "63aea7625dca5b139b9a60b4116312b2d7f60792cdf0371803996a89f11ff37",
    "Created": "2026-09-26T07:40:25.284144525Z",
    "Scope": "local",
    "Driver": "bridge",
    "EnableIPv4": true,
    "EnableIPv6": false,
    "IPAM": {
      "Driver": "default",
      "Options": {},
      "Config": [
        {
          "Subnet": "172.20.0.0/24",
          "Gateway": "172.20.0.1"
        }
      ]
    },
    "Internal": false,
    "Attachable": false,
    "Ingress": false,
    "ConfigFrom": {
      "Network": ""
    },
    "ConfigOnly": false,
    "Options": {
      "com.docker.network.enable_ipv4": "true",
      "com.docker.network.enable_ipv6": "false"
    },
    "Labels": {},
    "Containers": {},
    "Status": {
      "IPAM": {
        "Subnets": {
          "172.20.0.0/24": {
            "IPsInUse": 3,
            "DynamicIPsAvailable": 253
          }
        }
      }
    }
  }
]
```

docker network inspect devops\_net

## 2.3 Create Three Containers

Three Ubuntu-based containers were created:

| container | IP Address  | purpose             |
|-----------|-------------|---------------------|
| vm1       | 172.20.0.10 | Nginx reverse proxy |
| vm2       | 172.20.0.20 | Backend web server  |
| vm3       | 172.20.0.30 | Backend web server  |

The containers were configured to remain running using SSH as the container's main process.

Verify:

docker ps

```
adhit@LAPTOP-N3NM26QJ MINGW64 ~/docker-ansible-nginx-assessment
$ docker run -d \
  --name vm3 \
  --hostname vm3 \
  --network devops_net \
  --ip 172.20.0.30 \
  -p 127.0.0.1:2223:22 \
  --restart unless-stopped \
  devops-ubuntu:ssh-configured
51a34e30c893bdf444bb125bf9c1f92d2799a51db76416c496bb1a864bb62c0f

adhit@LAPTOP-N3NM26QJ MINGW64 ~/docker-ansible-nginx-assessment
$ docker ps --format "table {{.Names}}\t{{.Ports}}"
NAME          PORTS
vm3           127.0.0.1:2223->22/tcp
vm2           127.0.0.1:2222->22/tcp
vm1           127.0.0.1:2221->22/tcp
kind-cloud-provider 2375-2376/tcp
kind-registry-mirror
desktop-control-plane 127.0.0.1:53624->6443/tcp
desktop-worker

adhit@LAPTOP-N3NM26QJ MINGW64 ~/docker-ansible-nginx-assessment
$ ssh -p 2223 devops@localhost
The authenticity of host '[localhost]:2223 ([127.0.0.1]:2223)' can't be established.
ED25519 key fingerprint is: SHA256:ATpJeTBdqtmxWz5MVwGethYH9A+88F1ZnSaFkKoi9t0
This host key is known by the following other names/addresses:
  ~/.ssh/known_hosts:9: [localhost]:2221
  ~/.ssh/known_hosts:12: [localhost]:2222
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '[localhost]:2223' (ED25519) to the list of known hosts.
Welcome to Ubuntu 24.04.5 LTS (GNU/Linux 6.6.114.1-microsoft-standard-WSL2 x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

This system has been minimized by removing packages and content that are
not required on a system that users do not log into.

To restore this content, you can run the 'unminimize' command.

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

devops@vm3:~$ hostname
vm3
devops@vm3:~$ exit
logout
Connection to localhost closed.
```

SSH-vm3

```

adhit@LAPTOP-N3NM26QJ MINGW64 ~/docker-ansible-nginx-assessment
$ docker run -d \
  --name vm2 \
  --hostname vm2 \
  --network devops_net \
  --ip 172.20.0.20 \
  -p 127.0.0.1:2222:22 \
  --restart unless-stopped \
  devops-ubuntu:ssh-configured
5104fe2d894305303afc7c6d905e5bfb018dfc7ca288b8cf521c60ac173961e

adhit@LAPTOP-N3NM26QJ MINGW64 ~/docker-ansible-nginx-assessment
$ docker ps --format "table {{.Names}}\t{{.Ports}}"
TABLE
NAMES          PORTS
vm2            127.0.0.1:2222->22/tcp
vm1            127.0.0.1:2221->22/tcp
vm3
kind-cloud-provider 2375-2376/tcp
kind-registry-mirror
desktop-control-plane 127.0.0.1:53624->6443/tcp
desktop-worker

adhit@LAPTOP-N3NM26QJ MINGW64 ~/docker-ansible-nginx-assessment
$ ssh -p 2222 devops@localhost
The authenticity of host '[localhost]:2222 ([127.0.0.1]:2222)' can't be established.
ED25519 key fingerprint is: SHA256:ATpJeTBdqtmxWz5MVwGethYH9A+88F1ZnSaFkKoi9t0
This host key is known by the following other names/addresses:
~/.ssh/known_hosts:9: [localhost]:2221
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '[localhost]:2222' (ED25519) to the list of known hosts.
Welcome to Ubuntu 24.04.5 LTS (GNU/Linux 6.6.114.1-microsoft-standard-WSL2 x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

This system has been minimized by removing packages and content that are
not required on a system that users do not log into.

To restore this content, you can run the 'unminimize' command.

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

devops@vm2:~$ hostname
vm2
devops@vm2:~$ exit
logout
Connection to localhost closed.

```

## SSH-vm2

### 2.4 Automatic Restart

The containers use the unless-stopped restart policy.

Verify:

```
docker inspect -f '{{.Name}} -> RestartPolicy: {{.HostConfig.RestartPolicy.Name}}' vm1
vm2 vm3
```

Expected:

/vm1 -> RestartPolicy: unless-stopped

/vm2 -> RestartPolicy: unless-stopped

/vm3 -> RestartPolicy: unless-stopped

```

adhit@LAPTOP-N3NM26QJ MINGW64 ~/docker-ansible-nginx-assessment
$ docker inspect vm1 --format '{{.HostConfig.RestartPolicy.Name}}'
unless-stopped

adhit@LAPTOP-N3NM26QJ MINGW64 ~/docker-ansible-nginx-assessment
$ docker inspect vm2 --format '{{.HostConfig.RestartPolicy.Name}}'
unless-stopped

adhit@LAPTOP-N3NM26QJ MINGW64 ~/docker-ansible-nginx-assessment
$ docker inspect vm3 --format '{{.HostConfig.RestartPolicy.Name}}'
unless-stopped

```

## Restart policy

### 3. SSH Configuration and Hardening

OpenSSH Server was installed inside all three containers.

A dedicated user named devops was configured for SSH access.

SSH authentication uses an SSH key instead of a password.

The SSH configuration was hardened by:

- Disabling password authentication
- Disabling direct root login
- Allowing key-based authentication

SSH access was tested from the host.

Example:

```
ssh -p 2221 devops@localhost
```

After login:

hostname

Expected:vm1

```

adhit@LAPTOP-N3NM26QJ MINGW64 ~/docker-ansible-nginx-assessment
$ ssh -p 2221 devops@localhost "hostname"
ssh -p 2222 devops@localhost "hostname"
ssh -p 2223 devops@localhost "hostname"
vm1
vm2
vm3

adhit@LAPTOP-N3NM26QJ MINGW64 ~/docker-ansible-nginx-assessment
$ ssh -o PreferredAuthentications=password -o PubkeyAuthentication=no -p 2221 devops@localhost
devops@localhost: Permission denied (publickey).

adhit@LAPTOP-N3NM26QJ MINGW64 ~/docker-ansible-nginx-assessment
$ ssh -o PreferredAuthentications=password -o PubkeyAuthentication=no -p 2222 devops@localhost
devops@localhost: Permission denied (publickey).

adhit@LAPTOP-N3NM26QJ MINGW64 ~/docker-ansible-nginx-assessment
$ ssh -o PreferredAuthentications=password -o PubkeyAuthentication=no -p 2223 devops@localhost
devops@localhost: Permission denied (publickey).

adhit@LAPTOP-N3NM26QJ MINGW64 ~/docker-ansible-nginx-assessment
$ ssh -p 2221 root@localhost
root@localhost: Permission denied (publickey).

adhit@LAPTOP-N3NM26QJ MINGW64 ~/docker-ansible-nginx-assessment
$ ssh -p 2222 root@localhost
root@localhost: Permission denied (publickey).

adhit@LAPTOP-N3NM26QJ MINGW64 ~/docker-ansible-nginx-assessment
$ ssh -p 2223 root@localhost
root@localhost: Permission denied (publickey).

adhit@LAPTOP-N3NM26QJ MINGW64 ~/docker-ansible-nginx-assessment
$ |

```

## SSH security verification

### 3.1 Verify Password Login Is Rejected

Password authentication was explicitly tested:

```
ssh -o PreferredAuthentications=password \
```

```
-o PubkeyAuthentication=no \
```

```
-p 2221 devops@localhost
```

The connection was rejected with:

Permission denied (publickey).

This confirms that password authentication is disabled.

```
adhit@LAPTOP-N3NM26QJ MINGW64 ~/docker-ansible-nginx-assessment
$ docker exec vm2 sshd -T | grep -E 'passwordauthentication|permitrootlogin|pubkeyauthentication'
permitrootlogin no
pubkeyauthentication yes
passwordauthentication no

adhit@LAPTOP-N3NM26QJ MINGW64 ~/docker-ansible-nginx-assessment
$ docker exec vm3 sshd -T | grep -E 'passwordauthentication|permitrootlogin|pubkeyauthentication'
permitrootlogin no
pubkeyauthentication yes
passwordauthentication no

adhit@LAPTOP-N3NM26QJ MINGW64 ~/docker-ansible-nginx-assessment
$ docker exec vm3 sshd -T | grep -E 'passwordauthentication|permitrootlogin|pubkeyauthentication'
permitrootlogin no
pubkeyauthentication yes
passwordauthentication no

adhit@LAPTOP-N3NM26QJ MINGW64 ~/docker-ansible-nginx-assessment
$ |
```

Successful SSH login and rejected password authentication

## 4. Firewall Configuration

UFW was configured inside each VM with a default-deny incoming policy.

Check firewall status:

```
docker exec vm1 ufw status verbose
```

```
docker exec vm2 ufw status verbose
```

```
docker exec vm3 ufw status verbose
```

**vm1**

vm1 requires:

22/tcp

80/tcp

443/tcp

because it provides SSH, HTTP and HTTPS/reverse-proxy services.

**vm2 and vm3**

The backend servers require SSH and HTTP access from vm1.

Example:

22/tcp ALLOW IN Anywhere

80/tcp ALLOW IN 172.20.0.10

The default incoming policy is:

deny (incoming)

This restricts unnecessary inbound access.

```
adhit@LAPTOP-N3NM26QJ MINGW64 ~/docker-ansible-nginx-assessment
$ MSYS_NO_PATHCONV=1 docker exec vm1 ufw status verbose
MSYS_NO_PATHCONV=1 docker exec vm2 ufw status verbose
MSYS_NO_PATHCONV=1 docker exec vm3 ufw status verbose
Status: active
Logging: on (low)
Default: deny (incoming), allow (outgoing), deny (routed)
New profiles: skip

To Action From
--
22/tcp ALLOW IN Anywhere
80/tcp ALLOW IN Anywhere
443/tcp ALLOW IN Anywhere
22/tcp (v6) ALLOW IN Anywhere (v6)
80/tcp (v6) ALLOW IN Anywhere (v6)
443/tcp (v6) ALLOW IN Anywhere (v6)

Status: active
Logging: on (low)
Default: deny (incoming), allow (outgoing), deny (routed)
New profiles: skip

To Action From
--
22/tcp ALLOW IN Anywhere
22/tcp (v6) ALLOW IN Anywhere (v6)

Status: active
Logging: on (low)
Default: deny (incoming), allow (outgoing), deny (routed)
New profiles: skip

To Action From
--
22/tcp ALLOW IN Anywhere
22/tcp (v6) ALLOW IN Anywhere (v6)

adhit@LAPTOP-N3NM26QJ MINGW64 ~/docker-ansible-nginx-assessment
$ ssh -p 2221 devops@localhost "hostname"
ssh -p 2222 devops@localhost "hostname"
ssh -p 2223 devops@localhost "hostname"
vm1
vm2
vm3
```

UFW output for all three VMs

## 5. Local Domain Configuration

The following local domains were configured:

public.vm1.local

public.vm2.local

public.vm3.local

The Windows hosts file was updated:

C:\Windows\System32\drivers\etc\hosts

The domains were mapped to the corresponding container IP addresses.

Example:

172.20.0.10 public.vm1.local

172.20.0.20 public.vm2.local

172.20.0.30 public.vm3.local

This allows the local machine to resolve the .local hostnames without purchasing a domain.

## 6. TLS / HTTPS Configuration

A self-signed TLS certificate was generated for:

public.vm1.local

The certificate was configured in Nginx on vm1.

HTTP requests are redirected to HTTPS.

Test HTTP:

```
curl -I http://public.vm1.local:18080
```

The server returns an HTTP redirect.

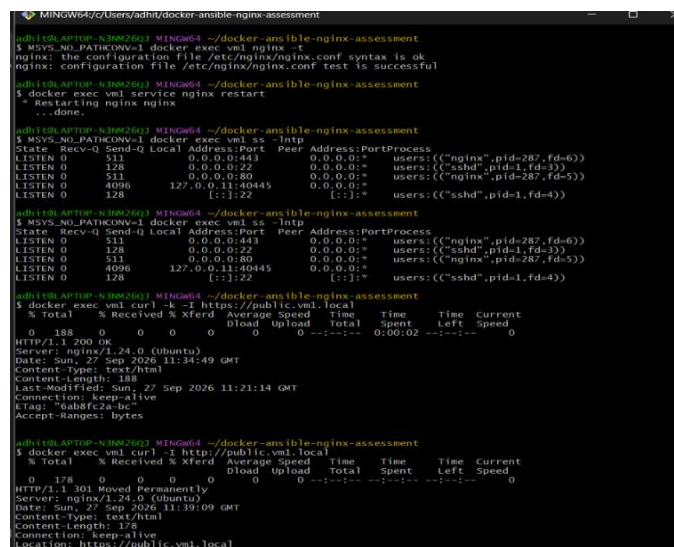
Test HTTPS:

```
curl -k -I https://public.vm1.local:18443
```

The HTTPS request returns:

HTTP/1.1 200 OK

The -k option is used because the certificate is self-signed.



```
adhi@UBUNTU:~/docker-ansible-nginx-assessment$ docker exec vm1 ss -ltnp
State Recv-Q Send-Q Local Address:Port Peer Address:PortProcess
LISTEN 0 128 0.0.0.0:443 0.0.0.0:* users:((("nginx",pid=287,fd=6))
LISTEN 0 128 0.0.0.0:22 0.0.0.0:* users:((("sshd",pid=1,fd=3))
LISTEN 0 512 0.0.0.0:80 0.0.0.0:* users:((("nginx",pid=287,fd=5))
LISTEN 0 4096 127.0.0.1:40445 0.0.0.0:* users:((("sshd",pid=1,fd=5))
LISTEN 0 128 :::22 :::* users:((("sshd",pid=1,fd=4))

adhi@UBUNTU:~/docker-ansible-nginx-assessment$ docker exec vm1 ss -ltnp
State Recv-Q Send-Q Local Address:Port Peer Address:PortProcess
LISTEN 0 128 0.0.0.0:443 0.0.0.0:* users:((("nginx",pid=287,fd=6))
LISTEN 0 128 0.0.0.0:22 0.0.0.0:* users:((("sshd",pid=1,fd=3))
LISTEN 0 512 0.0.0.0:80 0.0.0.0:* users:((("nginx",pid=287,fd=5))
LISTEN 0 4096 127.0.0.1:40445 0.0.0.0:* users:((("sshd",pid=1,fd=5))
LISTEN 0 128 :::22 :::* users:((("sshd",pid=1,fd=4))

adhi@UBUNTU:~/docker-ansible-nginx-assessment$ docker exec vm1 curl -I http://public.vm1.local
% Total % Received % Xferd Average Speed Time Time Time Current
0 0 0 0 0 0 0 0 0:00:02 --:--:-- 0
HTTP/1.1 200 OK
Server: nginx/1.24.0 (Ubuntu)
Date: Sun, 27 Sep 2026 11:34:49 GMT
Content-Type: text/html
Content-Length: 188
Last-Modified: Sun, 27 Sep 2026 11:21:14 GMT
Connection: keep-alive
Etag: "6abbfc2a-bc"
Accept-Ranges: bytes

adhi@UBUNTU:~/docker-ansible-nginx-assessment$ docker exec vm1 curl -k -I https://public.vm1.local
% Total % Received % Xferd Average Speed Time Time Time Current
0 0 0 0 0 0 0 0 0:00:02 --:--:-- 0
HTTP/1.1 301 Moved Permanently
Server: nginx/1.24.0 (Ubuntu)
Date: Sun, 27 Sep 2026 11:39:09 GMT
Content-Type: text/html
Content-Length: 178
Connection: keep-alive
Location: https://public.vm1.local
```

HTTP redirect and HTTPS response.

## 7. Ansible Setup

Ansible was installed in the Ubuntu WSL environment.

The Ansible project uses:

ansible/

```
|— ansible.cfg
|— inventory/
|   └─ hosts.ini
|— group_vars/
|   └─ all/
|       └─ vault.yml
|— roles/
|   └─ common/
|       └─ nginx/
|           └─ ssh/
└─ site.yml
```

## 8. Ansible Inventory

The inventory contains the three VMs:

vm1

vm2

vm3

Their Docker network IP addresses and SSH connection information are defined in the inventory.

Ansible connectivity was tested using:

```
ansible all -m ping
```

The result showed successful communication with:

vm1

vm2

vm3

```

ansible_user@vm1:~$ exit
logout
connection to 127.0.0.1 closed.
adhithya@APTOP-NINJA603:~/docker-ansible-nginx-assessment/ansible$ cd ~/docker-ansible-nginx-assessment/ansible
adhithya@APTOP-NINJA603:~/docker-ansible-nginx-assessment/ansible$ nano inventory/hosts.ini
adhithya@APTOP-NINJA603:~/docker-ansible-nginx-assessment/ansible$ ansible all -e ping
[WARNING]: Host 'vm2' is using the discovered python interpreter at '/usr/bin/python3.12', but future installation of another Python interpreter could cause a different interpreter to be discovered. See https://docs.ansible.com/ansible-core/2.20/reference_appendices/interpreter_discovery.html for more information.
all: hosts = [
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3.12"
  },
  "changed": false,
  "ping": "pong"
]
[WARNING]: Host 'vm3' is using the discovered python interpreter at '/usr/bin/python3.12', but future installation of another Python interpreter could cause a different interpreter to be discovered. See https://docs.ansible.com/ansible-core/2.20/reference_appendices/interpreter_discovery.html for more information.
all: hosts = [
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3.12"
  },
  "changed": false,
  "ping": "pong"
]
[WARNING]: Host 'vm1' is using the discovered python interpreter at '/usr/bin/python3.12', but future installation of another Python interpreter could cause a different interpreter to be discovered. See https://docs.ansible.com/ansible-core/2.20/reference_appendices/interpreter_discovery.html for more information.
all: hosts = [
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3.12"
  },
  "changed": false,
  "ping": "pong"
]
adhithya@APTOP-NINJA603:~/docker-ansible-nginx-assessment/ansible$

```

Ansible ping result.

## 9. Ansible Configuration

The ansible.cfg file defines the inventory and SSH private key.

Example:

[defaults]

inventory = inventory/hosts.ini

private\_key\_file = /home/adhithya/.ssh/id\_ed25519

host\_key\_checking = False

This avoids passing the SSH key manually on every Ansible command.

## 10. Ansible Roles

The playbook is divided into roles instead of using one large playbook.

### Common Role

The common role performs base system configuration such as:

- User creation
- Sudo configuration
- Required packages

### SSH Role

The SSH role handles:

- OpenSSH configuration
- SSH hardening
- Key-based authentication
- SSH service configuration

### Nginx Role

The Nginx role handles:

- Nginx installation
- Web page deployment
- Nginx configuration
- Reverse proxy configuration
- Nginx handlers

## 11. Ansible Vault

Sensitive information is stored using Ansible Vault.

The vault file:

`group_vars/all/vault.yml`

is encrypted.

The beginning of the file contains:

`$ANSIBLE_VAULT;1.2;AES256`

This prevents secrets from being stored as readable plaintext.

```
adhithya@LAPTOP-N3NM26QJ:~/docker-ansible-nginx-assessment/ansible$ head -n 1 group_vars/all/vault.yml
$ANSIBLE_VAULT;1.1;AES256
adhithya@LAPTOP-N3NM26QJ:~/docker-ansible-nginx-assessment/ansible$ |
```

Encrypted Vault file.

## 12. Ansible Playbook Execution

The main playbook is:

`site.yml`

It applies the required configuration to vm1, vm2 and vm3.

The playbook was executed using:

`ansible-playbook site.yml --ask-vault-pass`

## 13. Idempotency Test

The playbook was executed again after the initial configuration.

The second execution produced:

`changed=0`

`failed=0`

for the managed servers.

This demonstrates **idempotency**, meaning that running the same configuration again does not unnecessarily modify an already configured system.

```

ok: [vm1]
ok: [vm2]
ok: [vm3]

TASK [common : Verify vaulted secret is available] *****
ok: [vm2] => {
  "msg": "Vault secret loaded successfully"
}
ok: [vm1] => {
  "msg": "Vault secret loaded successfully"
}
ok: [vm3] => {
  "msg": "Vault secret loaded successfully"
}

TASK [ssh : Ensure SSH hardening configuration exists] *****
ok: [vm3]
ok: [vm2]
ok: [vm1]

TASK [nginx : Install Nginx] *****
ok: [vm3]
ok: [vm2]
ok: [vm1]

TASK [nginx : Ensure Nginx is running] *****
ok: [vm2]
ok: [vm1]
ok: [vm3]

TASK [nginx : Deploy dynamic Nginx page] *****
[DEPRECATION WARNING]: INJECT_FACTS_AS_VARS default to 'True' is deprecated, top-level facts will not be auto injected after the
change. This feature will be removed from ansible-core version 2.24.
Origin: /home/adhithya/docker-ansible-nginx-assessment/ansible/roles/nginx/templates/index.html.j2
Use 'ansible_facts["fact_name"]' (no 'ansible_' prefix) instead.
ok: [vm2]
ok: [vm3]
ok: [vm1]

TASK [nginx : Deploy Nginx reverse proxy configuration on vm1] *****
skipping: [vm2]
skipping: [vm3]
ok: [vm1]

PLAY RECAP *****
vm1      : ok=13   changed=0    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
vm2      : ok=12   changed=0    unreachable=0    failed=0    skipped=1    rescued=0    ignored=0
vm3      : ok=12   changed=0    unreachable=0    failed=0    skipped=1    rescued=0    ignored=0
adhithya@LAPTOP-N3NM26QJ:~/docker-ansible-nginx-assessment/ansible$

```

## Second playbook execution / PLAY RECAP

### 14. Nginx Installation and Dynamic Web Pages

Nginx was installed on all three VMs using Ansible.

Each VM serves a dynamically generated page containing:

- Hostname
- Current system time

The pages allow the backend server handling a request to be identified.

For example:

Server: vm2

Hostname: vm2

Current Time: ...

and:

Server: vm3

Hostname: vm3

Current Time: ...

## 15. Nginx Reverse Proxy

Nginx on vm1 acts as the reverse proxy.

The following routes were configured:

<https://public.vm1.local/vm2> → vm2

<https://public.vm1.local/vm3> → vm3

### vm2 Test

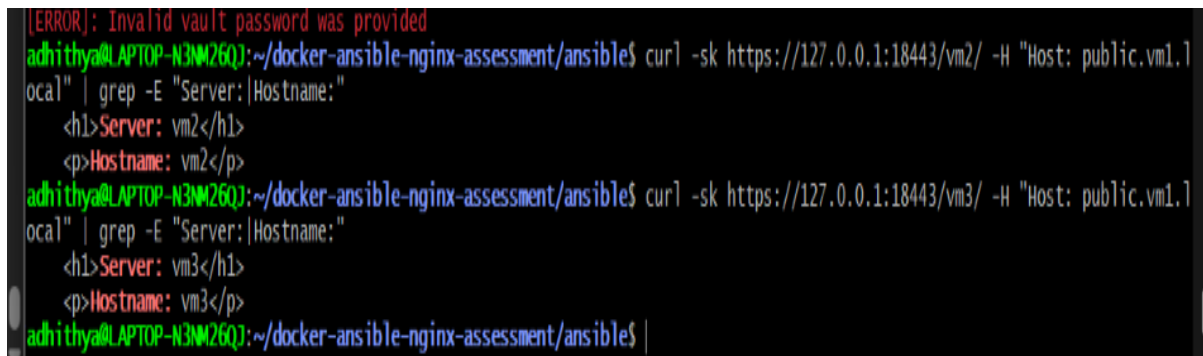
```
curl -k https://public.vm1.local:18443/vm2/
```

The response identifies vm2.

### vm3 Test

```
curl -k https://public.vm1.local:18443/vm3/
```

The response identifies vm3.



```
[ERROR]: Invalid vault password was provided
adithya@LAPTOP-N3M26QJ:~/docker-ansible-nginx-assessment/ansible$ curl -sk https://127.0.0.1:18443/vm2/ -H "Host: public.vm1.local" | grep -E "Server:|Hostname:"
<h1>Server: vm2</h1>
<p>Hostname: vm2</p>
adithya@LAPTOP-N3M26QJ:~/docker-ansible-nginx-assessment/ansible$ curl -sk https://127.0.0.1:18443/vm3/ -H "Host: public.vm1.local" | grep -E "Server:|Hostname:"
<h1>Server: vm3</h1>
<p>Hostname: vm3</p>
adithya@LAPTOP-N3M26QJ:~/docker-ansible-nginx-assessment/ansible$
```

/vm2 and /vm3 responses

## 16. Load Balancing

A third route was configured:

/app

Nginx uses an upstream configuration containing vm2 and vm3.

Multiple requests were sent:

for i in {1..6}; do

```
curl -sk https://public.vm1.local:18443/app/
```

done

The responses demonstrated that traffic can be served by both backend servers.

```

dhithya@LAPTOP-N3NM26QJ:~/docker-ansible-nginx-assessment/ansible$ curl -sk https://127.0.0.1:18443/vm2/ -H "Host: public.vm1.local" | grep -E "Server:|Hostname:"
<h1>Server: vm2</h1>
<p>Hostname: vm2</p>
dhithya@LAPTOP-N3NM26QJ:~/docker-ansible-nginx-assessment/ansible$ curl -sk https://127.0.0.1:18443/vm3/ -H "Host: public.vm1.local" | grep -E "Server:|Hostname:"
<h1>Server: vm3</h1>
<p>Hostname: vm3</p>
dhithya@LAPTOP-N3NM26QJ:~/docker-ansible-nginx-assessment/ansible$ docker ps --filter "name=vm2" --filter "name=vm3"
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS                    NAME
602a0c98b12   devops-ubuntu:ssh-configured        "/usr/sbin/sshd -D"     29 hours ago  Up 2 hours   127.0.0.1:2223->22/tcp    vm3
8f1735a2948   devops-ubuntu:ssh-configured        "/usr/sbin/sshd -D"     29 hours ago  Up 17 minutes 127.0.0.1:2222->22/tcp    vm2
dhithya@LAPTOP-N3NM26QJ:~/docker-ansible-nginx-assessment/ansible$ for i in {1..10}; do
echo "Request $i"
curl -sk https://127.0.0.1:18443/app/ -H "Host: public.vm1.local" | grep -E "Server:|Hostname:"
sleep 1
done
Request 1
<h1>Server: vm3</h1>
<p>Hostname: vm3</p>
Request 2
<h1>Server: vm2</h1>
<p>Hostname: vm2</p>
Request 3
<h1>Server: vm3</h1>
<p>Hostname: vm3</p>
Request 4
<h1>Server: vm2</h1>
<p>Hostname: vm2</p>
Request 5
<h1>Server: vm3</h1>
<p>Hostname: vm3</p>
Request 6
<h1>Server: vm2</h1>
<p>Hostname: vm2</p>
Request 7
<h1>Server: vm3</h1>
<p>Hostname: vm3</p>
Request 8
<h1>Server: vm2</h1>
<p>Hostname: vm2</p>
Request 9
<h1>Server: vm3</h1>
<p>Hostname: vm3</p>
Request 10
<h1>Server: vm3</h1>
<p>Hostname: vm3</p>
dhithya@LAPTOP-N3NM26QJ:~/docker-ansible-nginx-assessment/ansible$

```

/app requests showing vm2/vm3.

## 17. Failover Test

To demonstrate backend failure handling, vm2 was stopped:

docker stop vm2

Requests were then sent to /app:

for i in {1..6}; do

curl -sk https://public.vm1.local:18443/app/

done

During the failure test, requests continued to be served by vm3.

This demonstrates that traffic can continue through the available backend when vm2 becomes unavailable.

```

adhithya@LAPTOP-N3NM26QJ:~/docker-ansible-nginx-assessment/ansible$ for i in {1..10}; do
  echo "Request $i"
  curl -sk https://127.0.0.1:18443/app/ -H "Host: public.vm1.local" | grep -E "Server:|Hostname:"
  sleep 1
done
Request 1
<h1>Server: vm3</h1>
<p>Hostname: vm3</p>
Request 2
<h1>Server: vm3</h1>
<p>Hostname: vm3</p>
Request 3
<h1>Server: vm3</h1>
<p>Hostname: vm3</p>
Request 4
<h1>Server: vm3</h1>
<p>Hostname: vm3</p>
Request 5
<h1>Server: vm3</h1>
<p>Hostname: vm3</p>
Request 6
<h1>Server: vm3</h1>
<p>Hostname: vm3</p>
Request 7
<h1>Server: vm3</h1>
<p>Hostname: vm3</p>
Request 8
<h1>Server: vm3</h1>
<p>Hostname: vm3</p>
Request 9
<h1>Server: vm3</h1>
<p>Hostname: vm3</p>
Request 10
<h1>Server: vm3</h1>
<p>Hostname: vm3</p>
adhithya@LAPTOP-N3NM26QJ:~/docker-ansible-nginx-assessment/ansible$

```

vm2 stopped and /app requests continuing through vm3.

## 18. Backend Recovery

After the failover test, vm2 was restarted:

**docker start vm2**

The backend was then tested again:

for i in {1..6}; do

curl -sk https://public.vm1.local:18443/app/

done

After recovery, vm2 was available again as a backend without manually reloading Nginx.

```

adhithya@LAPTOP-N3NM26QJ:~/docker-ansible-nginx-assessment/ansible$ docker ps --filter "name=vm2" --filter
"name=vm3"
CONTAINER ID   IMAGE          NAMES                COMMAND                                CREATED        STATUS        PORTS
602a0c98b12   devops-ubuntu:ssh-configured  vm3                "/usr/sbin/sshd -D"                29 hours ago   Up About an hour   127.
.0.1:2223->22/tcp
8f1735a2948   devops-ubuntu:ssh-configured  vm2                "/usr/sbin/sshd -D"                29 hours ago   Up 10 seconds     127.
.0.1:2222->22/tcp
adhithya@LAPTOP-N3NM26QJ:~/docker-ansible-nginx-assessment/ansible$ for i in {1..10}; do
  echo "Request $i"
  curl -sk https://127.0.0.1:18443/app/ -H "Host: public.vm1.local" | grep -E "Server:|Hostname:"
  sleep 1
done
Request 1
<h1>Server: vm3</h1>
<p>Hostname: vm3</p>
Request 2
<h1>Server: vm3</h1>
<p>Hostname: vm3</p>
Request 3
<h1>Server: vm3</h1>
<p>Hostname: vm3</p>
Request 4
<h1>Server: vm3</h1>
<p>Hostname: vm3</p>
Request 5
<h1>Server: vm3</h1>
<p>Hostname: vm3</p>
Request 6
<h1>Server: vm3</h1>
<p>Hostname: vm3</p>
Request 7
<h1>Server: vm3</h1>
<p>Hostname: vm3</p>
Request 8
<h1>Server: vm3</h1>
<p>Hostname: vm3</p>
Request 9
<h1>Server: vm3</h1>
<p>Hostname: vm3</p>
Request 10
<h1>Server: vm3</h1>
<p>Hostname: vm3</p>
adhithya@LAPTOP-N3NM26QJ:~/docker-ansible-nginx-assessment/ansible$

```

vm2 running again and /app responses

## 19. Verification Summary

The following functionality was verified:

| Test                     | Result |
|--------------------------|--------|
| Three Docker VMs         | Passed |
| Custom Docker network    | Passed |
| Automatic restart policy | Passed |
| SSH key authentication   | Passed |
| Password SSH rejection   | Passed |
| UFW firewall             | Passed |
| Local domain resolution  | Passed |
| HTTPS                    | Passed |
| HTTP→ HTTPS redirect     | Passed |
| Ansible connectivity     | Passed |
| Ansible Vault            | Passed |
| Ansible idempotency      | Passed |
| Nginx web pages          | Passed |
| /vm2 reverse proxy       | Passed |
| /vm3 reverse proxy       | Passed |
| /app load balancing      | Passed |
| vm2 failover             | Passed |
| vm2 recovery             | Passed |

## 20. Project Repository

The complete project source code is maintained in the GitHub repository.

Repository structure:

docker-ansible-nginx-assessment/

├── ansible/

├── docker/

├── .gitignore

└── README.md

The repository contains the Docker configuration, Ansible inventory and roles, Vault configuration, Nginx configuration/templates, and project documentation.

Github repository link : <https://github.com/Adhithya20/docker-ansible-nginx-assessment>

## 21. Demonstration Video

The demonstration video covers:

- Docker containers
- Custom network
- Restart policy

- SSH key authentication
- Rejected password authentication
- Firewall configuration
- Ansible connectivity
- Ansible Vault
- Idempotency
- HTTPS
- Reverse proxy
- Load balancing
- Failover
- Backend recovery

**Demo Video link:**

[https://drive.google.com/file/d/19PboNXQluBWmiFr\\_8QnYBZu6yr2OsDPn/view?usp=sharing](https://drive.google.com/file/d/19PboNXQluBWmiFr_8QnYBZu6yr2OsDPn/view?usp=sharing)